Sauvegarder des données dans un jeu d'arcade

Notice développeur – scores, classements, progression…

Sur la borne, chaque jeu dispose d'un **espace de stockage personnel** dans la base de données. Tu y enregistres ce que tu veux (scores, noms, niveaux débloqués…) sous forme d'un tableau JSON, avec seulement **deux fonctions** :

```
const donnees = await lireDonneesJSON();   // lit ce qui a été sauvegardé (null si rien)
await ecrireDonneesJSON(donnees);          // sauvegarde (remplace tout le contenu précédent)
```

Quand ton jeu tourne sur ton ordinateur (sans la borne), les mêmes fonctions utilisent le **localStorage** du navigateur à la place : tu n'as rien à changer dans ton code entre ton PC et la borne.

1. Choisir le format de tes données

L'espace de ton jeu contient **une seule valeur JSON**. C'est à toi de décider de sa forme. Réfléchis d'abord à ce que tu veux garder, puis dessine la structure sur papier.

Exemple A – classement de scores : un tableau d'objets, un objet par partie.

```
[
  { "prenom": "Léa",  "nom": "Martin", "score": 1500 },
  { "prenom": "Hugo", "nom": "Durand", "score": 1200 }
]
```

Exemple B – progression : un objet avec plusieurs listes de ton choix.

```
{
  "niveauxDebloques": [1, 2, 3],
  "meilleursTemps": [42.5, 51.0, 38.2],
  "options": { "son": true, "difficulte": "facile" }
}
```

Exemple C – une simple liste :

```
["Léa", "Hugo", "Sam"]
```

Règles : utilise uniquement des nombres, textes, booleens (true/false), tableaux et objets. Pas de fonctions, ni d'objets Phaser (sprites, scènes…). Taille maximale : 10 Mo. Garde la même structure à chaque écriture, sinon ta lecture risque de planter.

Important : `ecrireDonneesJSON` **remplace tout**. Pour ajouter une entrée, il faut donc **lire** d'abord, **modifier**, puis **réécrire**.

2. Installer donnees.js dans ton jeu

Le fichier se trouve dans le dépôt de la borne : `app/static/js/donnees.js`. Copie-le dans le dossier `js` de ton jeu :

```
mon_jeu/
  index.js
  game.json
  js/
    donnees.js     <-- à copier ici
    menu.js
```

jeu.js

Tu peux le laisser dans ton jeu quand tu le livres. Sur la borne, la page charge déjà sa propre version et ta copie est ignorée : **il n'y a aucun conflit**.

3. Modifier index.js

Une seule modification : ajoute cette ligne **tout en haut** de ton `index.js`, avant les autres imports.

```
import "./js/donnees.js";      // <-- ligne à ajouter
import menu from "./js/menu.js";
import jeu from "./js/jeu.js";

var config = {
  type: Phaser.AUTO,
  width: 1280,
  height: 720,
  scene: [menu, jeu]
};

export var game = new Phaser.Game(config);
```

C'est tout : `lireDonneesJSON` et `ecrireDonneesJSON` sont maintenant utilisables dans toutes tes scènes, sans autre import.

4. Utiliser les fonctions dans tes scènes

Les deux fonctions sont **asynchrones** : il faut écrire `await` devant, dans une fonction déclarée `async`.

Lire (par exemple dans `create`) :

```
async create() {
  const scores = await lireDonneesJSON();
  if (scores === null) {
    // rien n'a encore été sauvegardé : on part d'une liste vide
  }
}
```

Ajouter un score (lire, modifier, réécrire) :

```
async function ajouterScore(prenom, nom, score) {
  let liste = await lireDonneesJSON();
  if (!Array.isArray(liste)) liste = [];       // premier lancement
  liste.push({ prenom: prenom, nom: nom, score: score });

  liste.sort((a, b) => b.score - a.score);     // meilleurs en premier
  liste = liste.slice(0, 10);                  // on garde le top 10

  await ecrireDonneesJSON(liste);
}
```

Afficher le classement :

```
async create() {
  const liste = (await lireDonneesJSON()) || [];
  liste.forEach((d, i) => {
    this.add.text(300, 150 + i * 40,
      (i + 1) + ". " + d.prenom + " " + d.nom + " - " + d.score,
      { fontSize: "28px", color: "#ffffff" });
  });
```

```
}
```

5. Tester

Sur ton ordinateur : lance ton jeu avec ton serveur local. Les données vont dans le localStorage. Pour les voir ou les effacer : outils développeur du navigateur (F12) > onglet *Application* (ou *Stockage*) > *Local Storage* > clé `arcade_donnees_<nom du dossier>`.

Sur la borne : les données sont enregistrées dans la base, sous le **nom du dossier de ton jeu**. Ne renomme donc pas ce dossier après avoir enregistré des données, sinon elles ne seront plus retrouvées. Deux jeux ne doivent pas porter le même nom de dossier.

Un jeu de démonstration complet est disponible dans `games/demos/applicationData` : inspire-toi de ses fichiers `index.js` et `js/ajouter.js`.

6. Erreurs fréquentes

Symptôme	Cause probable
window.lireDonneesJSON is undefined	L'import de donnees.js manque en haut de index.js, ou le chemin est faux.
Mon classement est écrasé	Tu as écrit sans lire avant : lis, modifie, puis réécris.
Cannot read properties of null	Rien n'est encore sauvegardé : teste if (liste === null) ou utilise || [].
Les données ont disparu sur la borne	Le dossier du jeu a été renommé.
Les fonctions renvoient une Promise	Il manque await (et async sur la fonction qui l'utilise).